

P1766R0

Mitigating minor modules maladies

Published Proposal, 2019-06-16

This version:

<http://wg21.link/p1766r0>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper identifies three minor issues relevant to modules and suggests resolutions

Table of Contents

1 Typedef names for linkage purposes

- 1.1 Compiler misbehavior
- 1.2 Modules impact
- 1.3 Proposed resolution
- 1.4 Rationale for fixing for C++20

2 Static assertions and empty declarations in `export`

- 2.1 Problem
- 2.2 Proposed resolution
- 2.3 Rationale for fixing for C++20

3 Default argument inconsistency

- 3.1 Problem
- 3.2 Proposal
- 3.3 Rationale for fixing for C++20

§ 1. Typedef names for linkage purposes

§ 1.1. Compiler misbehavior

```
template<typename T> int X;  
typedef struct {  
    int *n = &X<decltype(this)>;  
} Y;  
Y y;
```

GCC miscompiles this (incorrectly giving `X<Y>` internal linkage).

Clang rejects, giving a hint as to what's going on:

```
<source>:4:3: error: unsupported: typedef changes linkage of anonymous type  
} Y;  
^  
  
<source>:2:15: note: use a tag name here to establish linkage prior to defini  
typedef struct {  
    ^  
    Y
```

This is not an isolated example: lots of constructs within an anonymous struct can trigger a linkage computation, but the linkage computation cannot be performed until *after* we reach the end of the struct and find out whether it has a typedef name for linkage purposes.

The `typedef` keyword doesn't help us here. This case does not have a typedef name for linkage purposes:

```
typedef struct { /* ... */ } *p;
```

... and this case does:

```
struct { /* ... */ } typedef x; // ?!!
```

§ 1.2. Modules impact

When such a construct appears in a header unit, the problem is exacerbated. It is possible for the compiler to be aware of another definition of a struct from a different translation unit, but for that other definition to not be reachable:

```
// foo.h
#ifndef F00_H
#define F00_H
typedef struct { /*...*/ } X;
#endif
```

```
module;
#include "foo.h"
export module A;
X x;
```

```
export module B;
import A;
```

```
import B;
#include "foo.h"
```

In the final translation unit above, the definition of `X` is not (necessarily) reachable, so including a definition of `X` is acceptable. However, a compiler may already know of the definition of `X` in module `A`. This results in a need to "merge" the new definition with the old one. There are various techniques for this, but they typically rely on knowing that the definition is in fact a redeclaration of the already-known definition before anything "too complicated" happens.

§ 1.3. Proposed resolution

Typedef names for linkage purposes exist for C compatibility. Therefore it would be reasonable to restrict their use to C-compatible structures.

We propose disallowing any case where a class is given a typedef name for linkage and its members include anything other than:

- non-static data members with no default member initializers (and no *lambda-expressions* in the type of the member)
- `static_assert` declarations
- empty declarations

This is intended to ensure that structs with typedef names for linkage purposes are always simple enough that no linkage calculation is necessary before the typedef name for linkage is encountered, and likewise that the post-facto "merge" step is simple.

§ 1.4. Rationale for fixing for C++20

This is a bugfix; the issue is longstanding but is expected to become more significant in the presence of modules in C++20.

§ 2. Static assertions and empty declarations in `export`

§ 2.1. Problem

In the Modules TS and all subsequent proposals, there is a restriction that an `export` declaration or `export` block can contain only declarations that introduce at least one name.

For individual `export` declarations, this makes sense:

```
export static_assert(true); // error: what do you mean, 'export'?
```

But for `export` blocks, this rule introduces some awkwardness:

```

export {
    struct Foo { /*...*/ };
    static_assert(std::is_trivially_copyable_v<Foo>); // error: can't export

    struct Bar { /*...*/ };

    template<typename T> struct X { T t; };
    template<typename T> X(T) -> X<T>; // error: can't export this either!

    // ...

#define STR(x) constexpr char x[] = #x;
    STR(foo); // error: can't export empty-declaration ';'
    STR(bar); // error: can't export empty-declaration ';'
#undef X
}

```

To avoid this, the code must be reorganized: the `export` block could be split into pieces, or the `static_assert` and deduction guide could be moved elsewhere (and the redundant `;`s after the macros could be removed).

§ 2.2. Proposed resolution

Remove the rule that a declaration in an `export` block introduces at least one name. Retain the rule for the single-declaration form of `export`.

§ 2.3. Rationale for fixing for C++20

This fixes a usability problem with a new C++20 feature.

§ 3. Default argument inconsistency

§ 3.1. Problem

For non-inline function declarations, C++ permits default arguments to be different in different translation units. Likewise, for templates, C++ permits default template arguments to be different in different translation units:

```
// a.h  
int f(int a = 123);
```

```
// b.h  
int f(int a = 45);
```

It would be an error if both declarations were included into the same translation unit. But modules creates situations where both declarations may be simultaneously reachable (even though only one of them is visible) and it is unduly burdensome on an implementation to require that only the visible default argument is used, or even that the mismatch is detected and diagnosed.

```
import A;      // indirectly uses a.h, does not export it  
import "b.h";  
int n = f();   // ??
```

§ 3.2. Proposal

Disallow giving the same function parameter different default arguments in two declarations in the same namespace scope in different translation units, and likewise for default template arguments. No diagnostic is required unless one such function declaration is reachable where the other is encountered.

Note that we already have a similar rule for inline functions.

§ 3.3. Rationale for fixing for C++20

This fixes an area of underspecification and friction in a new C++20 feature.